

21

Computer Vision

Teaching machines to see and understand images

The big picture: from pixels to meaning

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	The field of making computers understand images
Short name	CV
Main tasks	Classify, detect, segment, and more
Main tools	CNNs and Vision Transformers
You've done it	Image Classifier + Visual Defect Detector

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is Computer Vision?	The field and why it's hard
2	The Four Main Tasks	Classify, detect, segment, keypoints
3	The CV Pipeline	From image to prediction
4	Image Preprocessing	Getting images ready
5	Data Augmentation	Free extra training data
6	CNNs vs Vision Transformers	Two ways to see
7	Transfer Learning for Vision	The standard shortcut
8	Real Applications	Where CV is used
9	A Vision Project in Code	Putting it together
10	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is Computer Vision?

Computer Vision (computer vision — computer vision), or CV, is the field of teaching computers to see and understand images and videos. The goal is to take in pixels and get out meaning — 'this is a cat', 'there are 3 people', 'this product has a scratch'.

It sounds simple because seeing is effortless for us. But it's genuinely hard for a computer. A photo is just a giant grid of numbers (remember the CNN topic). The same object can look totally different with changes in lighting, angle, size, or background. Computer Vision is about handling all that variation and still finding the meaning.

■ This is the field your Product Image Classifier and Visual Defect Detector live in. This topic zooms out to give you the full map of CV, so you can see where your projects fit.

2. The Four Main Tasks

Most computer vision work falls into four main tasks. They go from simple (one label for the whole image) to detailed (understanding every pixel).

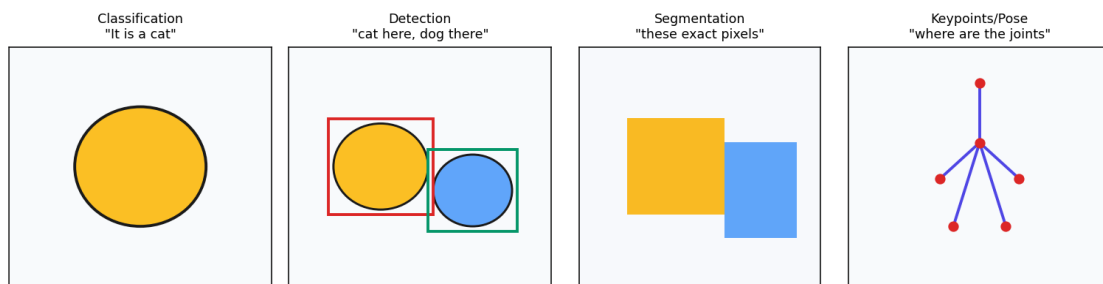


Fig 1 — The four main CV tasks, from simplest (classification) to most detailed (segmentation, keypoints).

Task	What it answers	Example
Classification	What is in this image?	'This is a cat' (one label)
Object Detection	What objects, and where?	Boxes around each cat and dog
Segmentation	Which exact pixels are what?	Outline the cat pixel-by-pixel
Keypoints / Pose	Where are the key points?	Find a person's joints

Classification (your image projects) gives one label for the whole picture. **Detection** finds and boxes multiple objects (next topic, with YOLO). **Segmentation** labels every pixel — useful for medical scans and self-driving cars. **Keypoints** find specific points, like body joints for fitness apps.

3. The Computer Vision Pipeline

Almost every CV project follows the same simple flow: take an image, prepare it, pass it through a model, get a prediction.

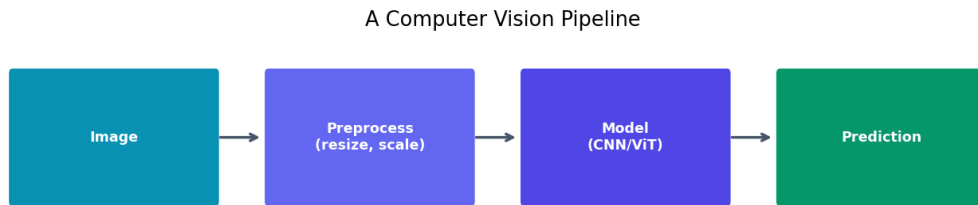


Fig 2 — The standard CV pipeline: image → preprocess → model → prediction.

Step	What happens
1. Image	Load the photo (from file, camera, etc.)
2. Preprocess	Resize, scale numbers, sometimes augment
3. Model	A CNN or Vision Transformer processes it
4. Prediction	Class, boxes, or pixel labels come out

4. Image Preprocessing

Before an image goes into a model, it needs preparation. Models expect images in a specific, consistent format. The two must-do steps:

- **Resize** — make every image the same size the model expects (e.g. 224×224 pixels).
- **Normalize** — scale the pixel numbers to a small range (like 0 to 1), so the model trains well.

```
from torchvision import transforms

preprocess = transforms.Compose([
    transforms.Resize((224, 224)),      # make all images same size
    transforms.ToTensor(),              # to a PyTorch tensor (0-1)
    transforms.Normalize(               # standard ImageNet values
        mean=[0.485, 0.456, 0.406],
        std =[0.229, 0.224, 0.225])
])
```

■ Those specific mean/std numbers are the ImageNet standard. When you use a model pre-trained on ImageNet (very common), you should normalize your images the same way it was trained.

5. Data Augmentation — Free Extra Data

Vision models need lots of images to learn well, but you don't always have many. **Data augmentation** (■■■■■ ■■■■■■■■■■■■) is a clever trick: make new training images by slightly changing the ones you have — flip, rotate, crop, or adjust brightness.

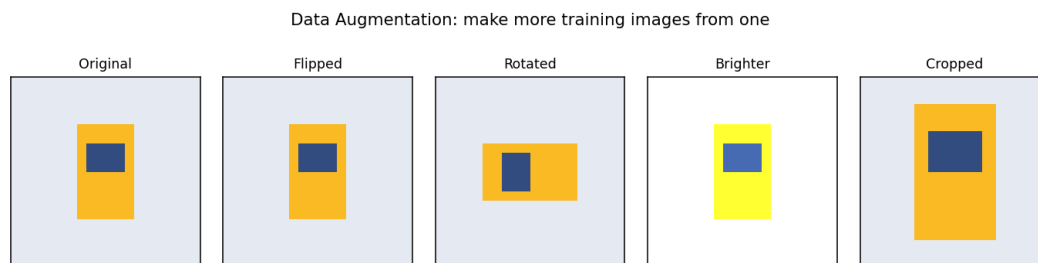


Fig 3 — From one image, augmentation makes many. A flipped cat is still a cat — more data for free.

A cat is still a cat whether it's flipped, rotated, or brighter. So each change gives the model a fresh example to learn from. This makes the model more robust (it sees more variety) and fights overfitting. It's one of the easiest ways to boost accuracy.

```
train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(),    # flip left-right
    transforms.RandomRotation(15),        # rotate a bit
    transforms.ColorJitter(brightness=0.2), # change brightness
    transforms.RandomResizedCrop(224),    # random crop + resize
    transforms.ToTensor(),
])
```

■ **Augment ONLY the training images, not the test images.** The test set should stay as-is so you measure real-world performance. This is a common point of confusion.

6. CNNs vs Vision Transformers

There are two main model types for vision today. **CNNs** (Topic 16) have been the workhorse for years. Newer **Vision Transformers** (ViT) apply the attention idea from Transformers to images by chopping them into patches and treating each patch like a word.

	CNN	Vision Transformer (ViT)
How it sees	Slides filters over the image	Splits image into patches, uses attention
Data needed	Works with less data	Needs lots of data to shine
Speed	Fast, efficient	Heavier, but very powerful at scale
Best for	Most everyday vision tasks	Very large datasets, top accuracy

Practical advice: for most projects, especially with limited data, a CNN (like your ResNet or EfficientNet) is the reliable choice. Vision Transformers shine when you have huge datasets. Both are excellent; CNNs remain the sensible default for everyday work.

7. Transfer Learning for Vision

This is the most important practical tip in all of computer vision: almost never train from scratch. Use **transfer learning** — start from a model already trained on millions of images (like ImageNet) and adapt it to your task. You covered this in the CNN topic; it's worth repeating because it's that important.

```
import torch.nn as nn
from torchvision import models

# Load a model pre-trained on millions of images
model = models.efficientnet_b0(weights='DEFAULT')

# Replace the final layer for your number of classes
model.classifier[1] = nn.Linear(
    model.classifier[1].in_features, 5)  # 5 classes

# Train on your (smaller) dataset - fast and accurate
```

■ *This is exactly the approach behind your Visual Defect Detector (EfficientNet-B0). Starting from a pre-trained model is why you reached high accuracy without millions of defect images.*

8. Real Applications

Field	What CV does there
Manufacturing	Spot product defects automatically (your project!)
Healthcare	Find tumours or issues in medical scans
Self-driving cars	See lanes, pedestrians, signs, other cars
Retail	Checkout-free stores, shelf monitoring
Security	Face recognition, anomaly detection
Agriculture	Spot crop disease from drone images
Phones	Unlock by face, sort photos, AR filters

Computer Vision is one of the most in-demand AI skills because it has so many real uses. Your defect detection project is a great portfolio piece precisely because manufacturing quality control is a real, valuable problem.

9. A Vision Project in Code

Here's the shape of a complete image classification project, tying the pieces together:

```
import torch, torch.nn as nn
from torchvision import models, transforms, datasets
from torch.utils.data import DataLoader

# 1. Preprocess + augment
train_tf = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomResizedCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                        [0.229, 0.224, 0.225])])

# 2. Load images from folders (one folder per class)
train_data = datasets.ImageFolder('data/train', train_tf)
loader = DataLoader(train_data, batch_size=32, shuffle=True)

# 3. Pre-trained model, swap final layer
model = models.resnet18(weights='DEFAULT')
model.fc = nn.Linear(model.fc.in_features, len(train_data.classes))

# 4. Train (same loop as the PyTorch topic)
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=0.001)
for epoch in range(10):
    for imgs, labels in loader:
        preds = model(imgs)
        loss = loss_fn(preds, labels)
        opt.zero_grad(); loss.backward(); opt.step()
```

10. Common Mistakes

Mistake	Why it's a problem	Fix
Not resizing images	Shape mismatch crash	Resize to model's expected size
Forgetting to normalize	Trains poorly	Use the right mean/std
Augmenting the test set	Unfair, wrong scores	Augment training only
Training from scratch	Needs huge data + time	Use transfer learning
Too little data, no augment	Overfits badly	Add data augmentation
Wrong normalize values	Pre-trained model confused	Match the model's training stats

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
Computer Vision	making computers understand images
Classification	one label for the whole image
Object detection	find + box multiple objects
Segmentation	label every pixel
Keypoints	find specific points (joints)
Resize	<code>transforms.Resize((224,224))</code>
Normalize	<code>transforms.Normalize(mean, std)</code>
Augmentation	<code>RandomFlip</code> , <code>RandomRotation</code> , <code>ColorJitter</code>
Load image folders	<code>datasets.ImageFolder(path, tf)</code>
Pre-trained CNN	<code>models.resnet18(weights='DEFAULT')</code>
Swap final layer	<code>model.fc = nn.Linear(in, n_classes)</code>
Vision Transformer	ViT — attention on image patches

The one idea to remember: Computer Vision turns pixels into meaning. Prepare the image (resize + normalize), optionally augment for more data, then use a pre-trained CNN or Vision Transformer and adapt it to your task with transfer learning.

Next Topic: Image Classification — a focused deep dive into the single most common CV task, the exact one behind your Product Image Classifier.